IKARUS 2026 • DATA SCIENCE CLUB

DATAQUEST — STAGE 2
DATA CHALLENGE

Participant Question Paper + Confidential Organizer Answer Key

Target audience: 2nd-year students • Recommended language: Python and SQL • Duration: 60 minutes • Total: 100 marks • Team size: 2–3 participants

Participant Instructions

1. Attempt all five problems. Total marks: 100.
2. Use Python for programming questions. For the SQL question, write a valid SQL query.
3. Read each input/output requirement carefully and follow the requested output format.
4. For data-analysis questions, show calculations clearly and state assumptions where needed.
5. Do not hard-code sample outputs. Your solution should work for other valid inputs.
6. Work only with your team. Copying another team's solution or using unauthorized assistance may lead to disqualification.
7. The evaluation panel's decision will be final.

Evaluation Rubric

Criterion	Weightage
Correctness and valid results	40%
Logic / analytical reasoning	25%
Code / query quality	15%
Efficiency	10%
Clear and complete submission	10%
Total	**100%**

PROBLEM 1 — PYTHON DATA CLEANING — REMOVE INVALID VALUES

Marks: 15 **Difficulty: Easy** **Concepts: Python lists, loops, filtering**

A sensor records daily temperatures. The value -999 means the reading is missing and must be ignored. Given N readings, print the number of valid readings and their sum.

Input Format

Line 1: integer N. Line 2: N space-separated integers.

Output Format

Print two lines: Valid readings: X and Sum: Y.

Constraints / Assumptions

1 ≤ N ≤ 1000. Readings are integers; -999 is reserved for missing values.

Sample Input	Expected Output
6\n25 27 -999 30 28 -999	Valid readings: 4\nSum: 110

PROBLEM 2 — STATISTICS — MEAN AND MEDIAN

Marks: 20 **Difficulty: Easy–Moderate** **Concepts: Sorting, mean, median**

Given N numerical observations, calculate the mean and median. For an even number of observations, the median is the average of the two middle values after sorting.

Input Format

Line 1: integer N. Line 2: N space-separated integers.

Output Format

Print Mean: X and Median: Y, each rounded to 2 decimal places.

Constraints / Assumptions

1 ≤ N ≤ 1000. Values are integers.

Sample Input	Expected Output
5\n2 4 6 8 10	Mean: 6.00\nMedian: 6.00
Sample Input	**Expected Output**
4\n1 3 7 9	Mean: 5.00\nMedian: 5.00

PROBLEM 3 — SQL — FIND HIGH-VALUE CUSTOMERS

Marks: 20 | **Difficulty: Moderate** | **Concepts: SQL SUM, GROUP BY, HAVING, ORDER BY**

A table named Sales has columns customer_id (integer), customer_name (text), and amount (numeric). Write an SQL query to show each customer's total purchase amount for customers whose total is at least 5000. Display customer_id, customer_name, and total_amount. Sort from highest total_amount to lowest.

Input Format

No console input. Write one SQL query using the Sales table.

Output Format

Return customer_id, customer_name, total_amount; group by customer and filter groups with total amount ≥ 5000; order descending by total_amount.

Constraints / Assumptions

Assume customer_id uniquely identifies each customer and amount is non-null.

Sample Input	Expected Output
Example rows: (1, Asha, 2000), (1, Asha, 3500), (2, Ravi, 1200), (3, Meera, 5000)	Expected rows:\n3 | Meera | 5000\n1 | Asha | 5500\n(Note: rows must be ordered by total_amount descending, so Asha appears before Meera.)

PROBLEM 4 — DATA INTERPRETATION — MONTHLY SALES

Marks: 20	Difficulty: Moderate	Concepts: Python, min/max, percentage change

A shop records the following monthly sales: 120, 150, 90, 180, and 210 units. Write a Python program that reads five space-separated sales values and prints the highest sales, lowest sales, and percentage increase from the first month to the fifth month. If the first month is zero, print Percentage increase: Undefined.

Input Format

One line containing five space-separated non-negative integers.

Output Format

Print Highest: X, Lowest: Y, Percentage increase: Z%. Round the percentage to 2 decimal places.

Constraints / Assumptions

Exactly five values. Percentage increase = ((last - first) / first) × 100.

Sample Input	Expected Output
120 150 90 180 210	Highest: 210\nLowest: 90\nPercentage increase: 75.00%
Sample Input	**Expected Output**
0 10 20 30 40	Highest: 40\nLowest: 0\nPercentage increase: Undefined

PROBLEM 5 — BASIC MACHINE LEARNING — CLASSIFICATION METRICS

Marks: 25 | **Difficulty: Moderate** | **Concepts: Confusion matrix, classification accuracy, arithmetic**

A binary classifier produces predictions for 10 cases. You are given four values: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). Calculate accuracy as (TP + TN) / (TP + TN + FP + FN) × 100 and print it to two decimal places.

Input Format

One line containing four integers in the order TP TN FP FN.

Output Format

Print Accuracy: X% with X rounded to 2 decimal places.

Constraints / Assumptions

All values are non-negative integers and their sum is greater than zero.

Sample Input	Expected Output
6 2 1 1	Accuracy: 80.00%
Sample Input	**Expected Output**
0 8 2 0	Accuracy: 80.00%

ORGANIZER-ONLY ANSWER KEY & HIDDEN TESTS

CONFIDENTIAL — DO NOT DISTRIBUTE TO PARTICIPANTS

Share only the participant section with competitors. Keep this answer key and the hidden tests for evaluation.

Problem 1 — Data Cleaning

```
n = int(input())
values = list(map(int, input().split()))
valid = [x for x in values if x != -999]
print("Valid readings:", len(valid))
print("Sum:", sum(valid))
```

Hidden Tests / Evaluation Notes

Input / Note	Expected Output / Evaluation
4\n-999 -999 5 10	Valid readings: 2\nSum: 15
3\n-999 -999 -999	Valid readings: 0\nSum: 0

Problem 2 — Mean and Median

```
n = int(input())
values = list(map(int, input().split()))
mean = sum(values) / n
values.sort()
if n % 2:
    median = float(values[n // 2])
else:
    median = (values[n // 2 - 1] + values[n // 2]) / 2
print(f"Mean: {mean:.2f}")
print(f"Median: {median:.2f}")
```

Hidden Tests / Evaluation Notes

Input / Note	Expected Output / Evaluation
4\n10 10 10 10	Mean: 10.00\nMedian: 10.00
3\n-5 0 5	Mean: 0.00\nMedian: 0.00

Problem 3 — SQL

```
SELECT customer_id, customer_name, SUM(amount) AS total_amount
FROM Sales
GROUP BY customer_id, customer_name
HAVING SUM(amount) >= 5000
ORDER BY total_amount DESC;
```

Hidden Tests / Evaluation Notes

Input / Note	Expected Output / Evaluation
Validation note	Check that GROUP BY, HAVING, SUM and descending order are present.

Problem 4 — Monthly Sales

```
sales = list(map(int, input().split()))
first, last = sales[0], sales[-1]
print("Highest:", max(sales))
print("Lowest:", min(sales))
if first == 0:
    print("Percentage increase: Undefined")
else:
    change = ((last - first) / first) * 100
    print(f"Percentage increase: {change:.2f}%")
```

Hidden Tests / Evaluation Notes

Input / Note	Expected Output / Evaluation
100 100 100 100 100	Highest: 100\nLowest: 100\nPercentage increase: 0.00%
200 180 160 140 100	Highest: 200\nLowest: 100\nPercentage increase: -50.00%

Problem 5 — Accuracy

```
tp, tn, fp, fn = map(int, input().split())
total = tp + tn + fp + fn
accuracy = (tp + tn) / total * 100
print(f"Accuracy: {accuracy:.2f}%")
```

Hidden Tests / Evaluation Notes

Input / Note	Expected Output / Evaluation
10 0 0 0	Accuracy: 100.00%
0 0 5 5	Accuracy: 0.00%

Score Summary & Tie-Breaking

Problem	Topic	Marks
1	Python data cleaning	15
2	Mean and median	20
3	SQL aggregation	20
4	Sales interpretation	20
5	Classification accuracy	25
TOTAL		**100**

Suggested tie-break order: (1) correctness on hidden tests, (2) score on Problem 5, (3) earliest complete submission. Confirm and announce this rule before the event.

Organizer Checklist

- Distribute participant pages only; withhold organizer answer key.
- Use the same hidden tests for every team.
- For SQL, specify the SQL dialect if an online platform is used.
- For analytical questions, award method marks for correct formulas and clear reasoning.
- Record scores out of 100 and announce the qualification rule before the round.